

Технологии программирования

Лекция 7

Модель исключений Python. Модули.

**ПЕРЕЛОГИНЬТЕСЬ С ФАМИЛИЕЙ И ИМЕНЕМ!
АКТИВНОСТЬ БУДЕТ СЧИТАТЬСЯ, ОПИРАЯСЬ НА ЧАТ**

Обо мне

**Кирилл
Сысоев**

6+ лет в Big Data
HSE University
Lead Data Engineer
Sber

Hadoop, Spark, ClickHouse, Kafka, Kubernetes
Python/Scala, SQL



t.me/KRSysoev
ksysoev@hse.ru

ТG-канал
«Кузница данных»

Темы лекций

1-2) Синтаксис Python

3-4) ООП и концепции тестирования

5-6) Bash и Git

7) Модули, файловая система и исключения в Python

8-9) Docker

10-11) BigData и Hadoop

12) DevOps

Цель лекции

1. Разобраться, как Python позволяет обрабатывать ошибки
2. Узнать, что такое исключения
3. Познакомиться с понятием модуля Python

Исключения

Напишем программу, которая считает обратные значения для целых чисел из заданного диапазона и выводит их в одну строку с разделителем «;»

```
print(";".join(str(1 / x) for x in range(int(input()), int(input()) + 1)))
```

В чем проблема такой реализации?

Проблема: Если в диапазоне окажется число 0 (например, от -1 до 1), возникнет ошибка:

```
print(";".join(str(1 / x) for x in range(int(input()), int(input()) + 1)))
```

```
ZeroDivisionError: division by zero
```

Ошибка прерывает выполнение программы — она называется исключением.

Исключения в Python — это события, которые происходят при нарушении корректного выполнения кода и останавливают его работу.

Попробуем избежать ошибки деления на ноль, добавив проверку:

```
1 interval = range(int(input()), int(input()) + 1)
2 if 0 in interval:
3     print("Диапазон чисел содержит 0.")
4 else:
5     print(":".join(str(1 / x) for x in interval))
```

```
1 interval = range(int(input()), int(input()) + 1)
2 if 0 in interval:
3     print("Диапазон чисел содержит 0.")
4 else:
5     print(";".join(str(1 / x) for x in interval))
```

Теперь программа работает безопасно для диапазонов вроде от -2 до 2. Однако она по-прежнему может завершиться ошибкой, если ввести не числа:

```
ValueError: invalid literal for int() with base 10: 'a'
```

Добавим проверку корректности ввода заранее. Для этого используем метод `.isdigit()` и учтём возможность отрицательных чисел с помощью `.lstrip("-")`:

```
1  start = input()
2  end = input()
3  if not (start.lstrip("-").isdigit() and end.lstrip("-").isdigit()):
4      print("Необходимо ввести два числа.")
5  else:
6      interval = range(int(start), int(end) + 1)
7      if 0 in interval:
8          print("Диапазон чисел содержит 0.")
9      else:
10         print(":".join(str(1 / x) for x in interval))
```

Программа работает устойчиво и не вызывает исключения даже при ошибочном вводе!

**В чём разница между подходами
LBUL и EAFR
при работе с ошибками**

Подход, который был нами применён, называется Look Before You Leap (LBYL), или **«Сначала проверь, потом действуй»**.

В программе проверяются возможные условия возникновения ошибок до исполнения основного кода.

У подхода LBYL есть свои недостатки:

- Стало сложнее читать из-за вложенного условного оператора.
- Проверка условия, что строка может быть преобразована в число, выглядит даже сложнее, чем само выражение.

Кроме того, вложенный условный оператор не решает поставленную задачу, а только лишь проверяет входные данные на корректность. Легко заметить, что решение основной задачи заняло меньше времени, чем составление условий проверки корректности входных данных.

Существует другой подход для работы с ошибками: Easier to Ask Forgiveness than Permission (EAFP), или «**Проще попросить прощения, чем разрешения**».

В этом подходе сначала исполняется код, а в случае возникновения ошибок происходит их обработка.

Подход EAFP реализован в Python в виде механизма обработки исключений.

Параметр сравнения	LBYL (Look Before You Leap)	EAFP (Easier to Ask Forgiveness than Permission)
Идея	Сначала проверяем, можно ли выполнить действие	Сначала пытаемся выполнить, а потом обрабатываем ошибки
Характер кода	Защитный, с большим числом условий	Оптимистичный, с обработкой ошибок постфактум
Минусы	Много условий, код перегружен проверками	Возможны неожиданные ошибки, если не учесть исключения
Типичный сценарий	Возможные ошибки малочисленны или не требуют сложной проверки	Ошибки возникают на разных уровнях вложенности кода, но приводят к равноценным вариантам обработки

**Как работает конструкция try-except и
когда использовать else и finally**

Чтобы обрабатывать ошибки в Python, используется конструкция try–except. Она позволяет «поймать» исключение, если оно произошло, и выполнить нужные действия — вместо того чтобы программа аварийно завершилась.

```
1  try:
2      <код, который может вызвать исключение>
3  except <КлассИсключения_1>:
4      <обработка ошибки этого типа>
5  except <КлассИсключения_2>:
6      <обработка другого типа ошибки>
7      ...
8  else:
9      <выполняется, если исключений не было>
10 finally:
11     <выполняется всегда -- и при ошибках, и без>
```

Порядок блоков `except` **важен!**

Когда в блоке `try` возникает ошибка – Python начинает искать подходящий блок `except`, чтобы её обработать.
Он делает это сверху вниз, по порядку.

Если первым стоит `except Exception`, то он перехватит практически любое исключение (почти все встроенные ошибки наследуются от `Exception`).

В этом случае интерпретатор не дойдёт до остальных блоков,
и они просто не выполнятся.

Это означает, что более специфические обработчики окажутся бесполезными.

Пример 1. Правильный порядок

```
1  try:
2      print(1 / int(input()))
3  except ZeroDivisionError:
4      print("Ошибка деления на ноль.")
5  except ValueError:
6      print("Невозможно преобразовать строку в число.")
7  except Exception:
8      print("Неизвестная ошибка.")
```

Если ввести «0», будет выведено: Ошибка деления на ноль.

Если ввести «a», будет выведено: Невозможно преобразовать строку в число.

Если произойдёт другая ошибка — сработает except Exception.

Пример 2. Неправильный порядок

```
1  try:
2      print(1 / int(input()))
3  except Exception:
4      print("Неизвестная ошибка.")
5  except ZeroDivisionError:
6      print("Ошибка деления на ноль.")
7  except ValueError:
8      print("Невозможно преобразовать строку в число.")
```

Здесь всегда сработает первый блок `except Exception`, и остальные никогда не будут выполнены. Поэтому при вводе любого некорректного значения программа выведет только: Неизвестная ошибка.

Чтобы обработчики работали корректно, располагайте их **от частных к общим** — в порядке убывающей специфичности!

Блок else

else используется, если вы хотите выполнить действия только в случае, если ошибок не было.

Это удобно для явного разделения логики и обработки ошибок:

```
1  try:
2      print(1 / int(input()))
3  except ZeroDivisionError:
4      print("Ошибка деления на ноль.")
5  except ValueError:
6      print("Невозможно преобразовать строку в число.")
7  except Exception:
8      print("Неизвестная ошибка.")
9  else:
10     print("Операция выполнена успешно.")
```

Ввод 5:

```
0.2
Операция выполнена успешно.
```

Ввод 0:

```
Ошибка деления на ноль.
```

Блок finally

Блок `finally` всегда выполняется, независимо от того, было ли исключение. Его используют для завершения операций (например, чтобы закрыть файл или освободить ресурсы):

```
1  try:
2      print(1 / int(input()))
3  except ZeroDivisionError:
4      print("Ошибка деления на ноль.")
5  except ValueError:
6      print("Невозможно преобразовать строку в число.")
7  except Exception:
8      print("Неизвестная ошибка.")
9  else:
10     print("Операция выполнена успешно.")
11 finally:
12     print("Программа завершена.")
```

Подход LBYL

```
1 start = input()
2 end = input()
3 if not (start.lstrip("-").isdigit() and end.lstrip("-").isdigit()):
4     print("Введите два числа.")
5 else:
6     interval = range(int(start), int(end) + 1)
7     if 0 in interval:
8         print("Диапазон чисел содержит 0.")
9     else:
10        print(";".join(str(1 / x) for x in interval))
```

Что лучше?

Подход EAFP

```
1 try:
2     print(";".join(str(1 / x) for x in range(int(input()), int(input()))
3 except ZeroDivisionError:
4     print("Диапазон чисел содержит 0.")
5 except ValueError:
6     print("Необходимо ввести два числа.")
```

Подход LBYL

```
1 start = input()
2 end = input()
3 if not (start.lstrip("-").isdigit() and end.lstrip("-").isdigit()):
4     print("Введите два числа.")
5 else:
6     interval = range(int(start), int(end) + 1)
7     if 0 in interval:
8         print("Диапазон чисел содержит 0.")
9     else:
10        print(";".join(str(1 / x) for x in interval))
```

- Код короче
- Проще читается

Подход EAFP

```
1 try:
2     print(";".join(str(1 / x) for x in range(int(input()), int(input()))
3 except ZeroDivisionError:
4     print("Диапазон чисел содержит 0.")
5 except ValueError:
6     print("Необходимо ввести два числа.")
```

Оператор raise

Иногда нужно не просто обрабатывать исключения, но и вызывать их вручную, если данные некорректны или программа зашла в «опасное» состояние.

Для этого используется оператор raise:

`raise <КлассИсключения>(сообщение)`

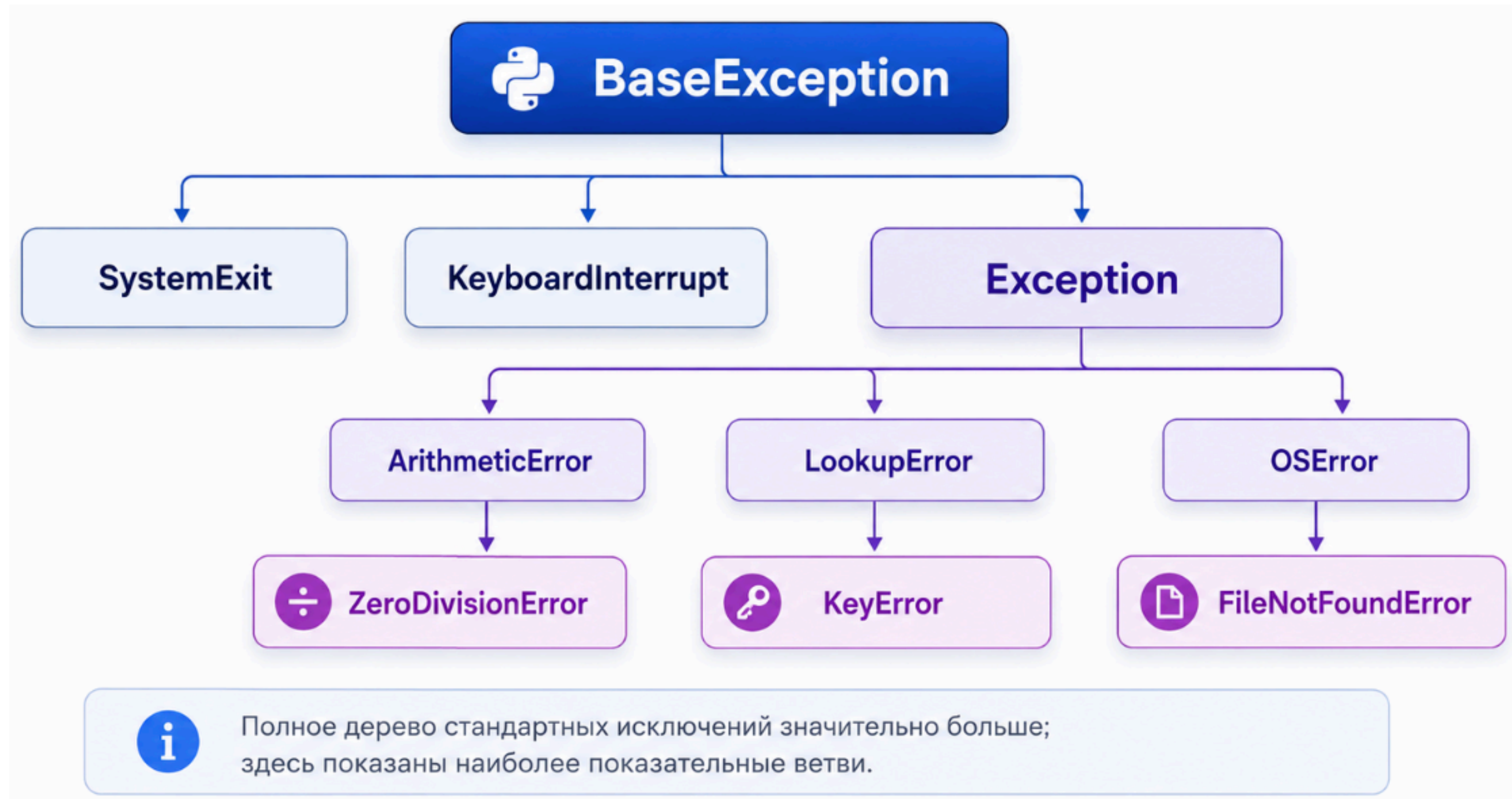
```
1  x = int(input("Введите число: "))
2  if x < 0:
3      raise ValueError("Число должно быть положительным")
```

Иерархия исключений в Python

Исключения — это классы, можно использовать встроенные и создавать собственные. Такие классы могут наследоваться от `Exception` или любого другого подходящего базового исключения.

Исключения в Python являются объектами классов ошибок. Сами классы исключений организованы в иерархию, где все стандартные исключения наследуются от базового класса `BaseException`. Благодаря этому механизму можно группировать исключения и обрабатывать их гибко и последовательно.

Иерархия исключений в Python



Классов исключений в Python 3.10.8 около 65!

Собственное исключение

В Python можно создавать собственные классы исключений. Это удобно, если вы хотите точно описывать, какая именно ошибка произошла в вашей программе.

Создавать исключения следует через наследование от стандартных классов, чаще всего от Exception.

Создадим программу, которая складывает список целых чисел, но выбрасывает исключения, если в списке есть хотя бы одно чётное или отрицательное число.

Для этого мы создадим:

- `NumbersError` — базовый класс для ошибок, связанных с числами
- `EvenError` — пользовательское исключение, которое мы будем выбрасывать, если в списке есть хотя бы одно чётное число
- `NegativeError` — пользовательское исключение, которое выбрасывается, если есть хотя бы одно отрицательное число

Собственное исключение

```
1  class NumbersError(Exception):
2      pass
3
4  class EvenError(NumbersError):
5      pass
6
7  class NegativeError(NumbersError):
8      pass
9
10 def no_even(numbers):
11     if all(x % 2 != 0 for x in numbers):
12         return True
13     raise EvenError("В списке не должно быть чётных чисел")
14
15 def no_negative(numbers):
16     if all(x >= 0 for x in numbers):
17         return True
18     raise NegativeError("В списке не должно быть отрицательных чисел")
19
20 def main():
21     print("Введите числа в одну строку через пробел:")
22     try:
23         numbers = [int(x) for x in input().split()]
24         if no_negative(numbers) and no_even(numbers):
25             print(f"Сумма чисел равна: {sum(numbers)}.")
26     except NumbersError as e: # Обращение к исключению как к объекту
27         print(f"Произошла ошибка: {e}.")
28     except Exception as e:
29         print(f"Произошла непредвиденная ошибка: {e}.")
30
31 if __name__ == "__main__":
32     main()
```

Исключения – удобно

- Исключения стали наглядными и осмысленными: из текста ошибки понятно, что именно пошло не так.
- Мы можем разделять обработку разных типов ошибок — и, если нужно, по-разному на них реагировать.
- Такой подход помогает писать надёжный и предсказуемый код, особенно в сложных проектах.

Модуль

Модуль

Каждая программа на Python — это потенциальный модуль, который можно импортировать в другие файлы. Это мощный инструмент повторного использования кода.

Запуск файла напрямую

```
1  if __name__ == "__main__":  
2      main()
```

Если файл запущен напрямую, `__name__` будет равен `"__main__"` и блок кода внутри `if` выполнится.

Если же файл импортируют как модуль — этот код пропускается.

Как правильно импортировать модули

Есть два способа импортировать модуль:

1. Импортировать целиком (всё пространство имён).
2. Импортировать отдельные объекты.

```
1 import example_module
```

```
1 example_module.some_function()
```

```
1 from example_module import some_function, ExampleClass
```

Пример бага

У нас есть файл `module_hello.py` с таким содержанием:

```
1  def hello(name):  
2      return f"Привет, {name}!"  
3  
4  print(hello(input("Введите своё имя: ")))
```

И есть вторая программа `program.py`, в которой мы хотим использовать функцию `hello()`:

```
1  from module_hello import hello  
2  
3  print(hello(input("Добрый день. Введите имя: ")))
```

Если запустить `program.py`, вы получите два запроса имени подряд.

Потому что при импорте `module_hello` Python выполнил весь код в файле, включая `print(hello(...))`. То есть программа начала работать ещё до того, как дошла до строки `print(...)` в `program.py`.

Исправление бага

```
1  def hello(name):  
2      return f"Привет, {name}!"  
3  
4  if __name__ == "__main__":  
5      print(hello(input("Введите своё имя: ")))
```

```
1  def hello(name):  
2      return f"Привет, {name}!"  
3  
4  def main():  
5      print(hello(input("Введите своё имя: ")))  
6  
7  if __name__ == "__main__":  
8      main()
```

Теперь при импорте ничего не будет выполнено.

Для большей читаемости основную часть программы часто оформляют как отдельную функцию `main()`:

Импорт всего

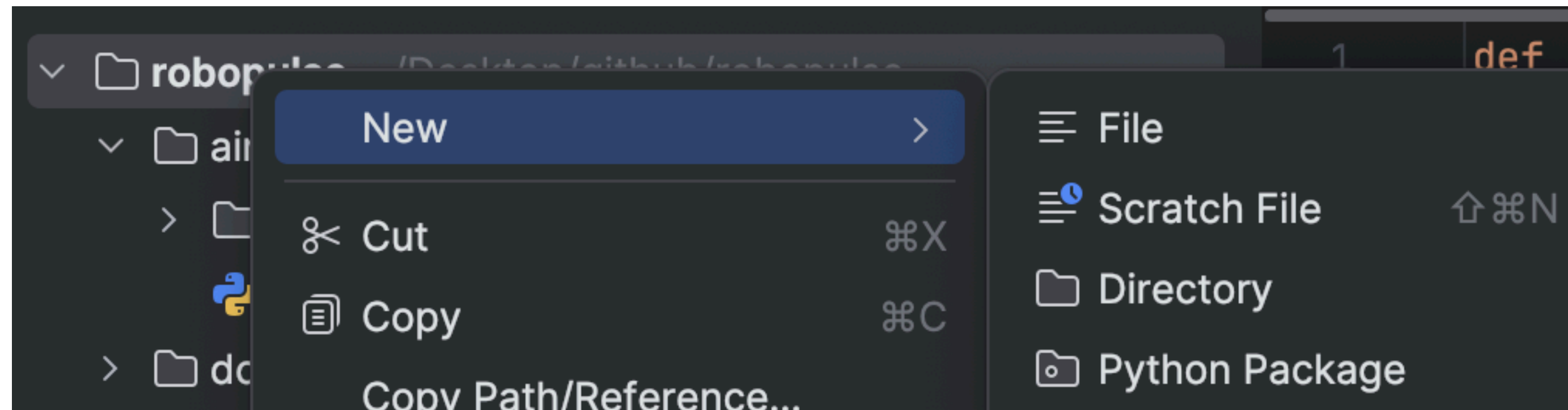
```
1 from some_module import *
```

Импортировать все не рекомендуется!

- Это загрязняет пространство имён: вы не видите, откуда взялась переменная `x` или функция `run()`.
- Если в вашем файле уже есть объект с таким именем — он будет перезаписан.
- Это затрудняет чтение и отладку кода.

Как правильно добавлять код в PyCharm

Хотим создать 2 файла в директориях



При абсолютном пути все работает

```
Project ▾  
  ▾ robopulse ~/Desktop/github/robopulse  
    ▾ airflow  
      > dags  
        test_import.py  
      > docker  
      > docs  
      > infra  
      > jupyter  
      > scripts  
      > spark  
      ▾ src  
        ▾ hi_scripts  
          hello_methods.py  
      > venv  
        .gitignore  
        CODEX_PROJECT_CONTEXT.md  
        Dockerfile.example  
        generate_data.py  
        Makefile  
        process_silver.py  
        README.md  
        requirements.txt  
      > External Libraries  
  
hello_methods.py ×  
1 def hi1(person_name): 2 usages  
2     print(f'Зануьк hello_methods {__name__}')  
3     return f'Hello, {person_name}'  
4  
5  
6     print(f'Зануьк импортированного файла {__name__}')  
7  
  
test_import.py ×  
1 from src.hi_scripts.hello_methods import hi1  
2  
3  
4     print(f'Зануьк обычного файла {__name__}')  
5  
6     print(f'Зануьк test_import {__name__}')  
7     print(hi1('Kirill'))  
8  
  
Run test_import ×  
/Users/ksysoev/Desktop/github/robopulse/venv/bin/python /Users/ksysoev/Desktop/github/robopulse/airflow/test_import.py  
Зануьк импортированного файла src.hi_scripts.hello_methods  
Зануьк обычного файла __main__  
Зануьк test_import __main__  
Зануьк hello_methods src.hi_scripts.hello_methods  
Hello, Kirill  
Process finished with exit code 0
```

Если импортируем из пакета (относительный путь) – ошибка

The screenshot shows an IDE with a project named 'robopulse' located at '~/.Desktop/github/robopulse'. The project structure includes folders like 'airflow', 'dags', 'docker', 'docs', 'infra', 'jupyter', 'scripts', 'spark', 'src', and 'venv'. Inside 'src', there is a folder 'hi_scripts' containing 'hello_methods.py'. The 'venv' folder is selected, showing files like '.gitignore', 'CODEX_PROJECT_CONTEXT.md', 'Dockerfile.example', 'generate_data.py', 'Makefile', 'process_silver.py', 'README.md', and 'requirements.txt'.

Two Python files are open in the editor:

- hello_methods.py**:

```
1 def hi1(person_name):  
2     print(f'Заняк hello_methods {__name__}')  
3     return f'Hello, {person_name}'  
4  
5  
6 print(f'Заняк импортированного файла {__name__}')7
```
- test_import.py**:

```
1 from hi_scripts.hello_methods import hi1  
2  
3  
4 print(f'Заняк обычного файла {__name__}')5  
6 print(f'Заняк test_import {__name__}')7 print(hi1('Kirill'))  
8
```

The 'Run' tab shows the execution of 'test_import.py' using the command: `/Users/ksysoev/Desktop/github/robopulse/venv/bin/python /Users/ksysoev/Desktop/github/robopulse/airflow/test_import.py`. The output is a traceback error:

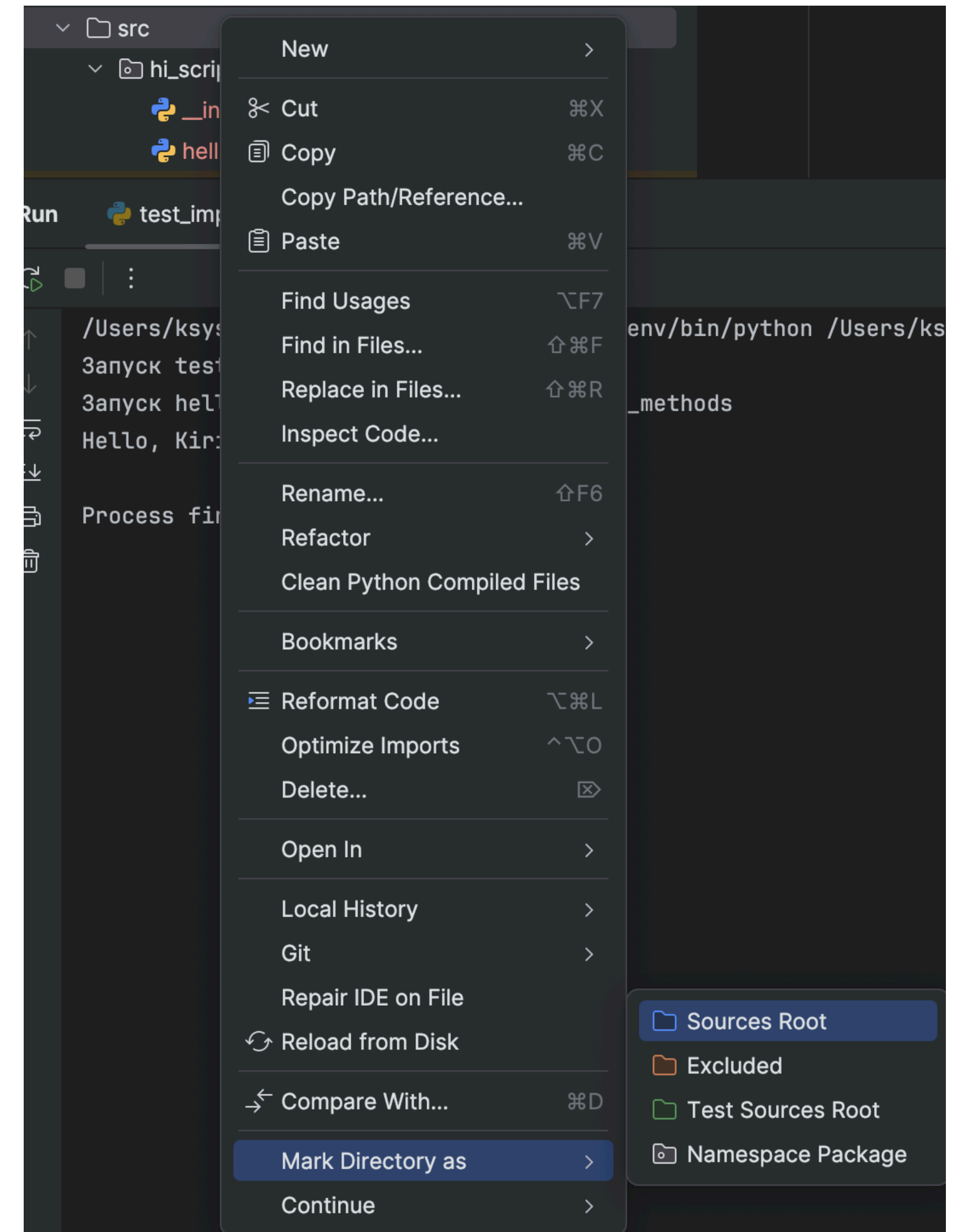
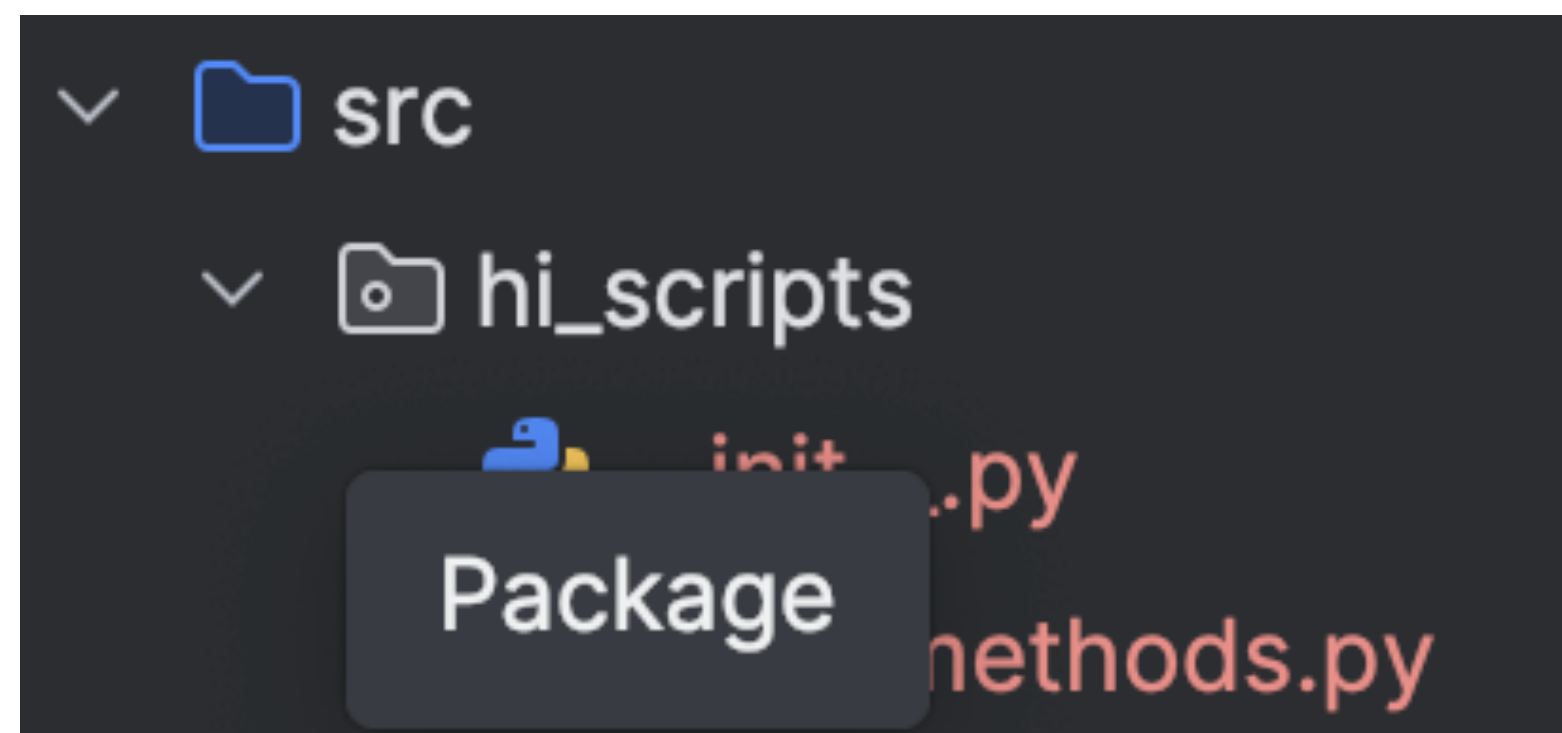
```
Traceback (most recent call last):  
  File "/Users/ksysoev/Desktop/github/robopulse/airflow/test_import.py", line 1, in <module>  
    from hi_scripts.hello_methods import hi1  
ModuleNotFoundError: No module named 'hi_scripts'
```

The process finished with exit code 1.

Решение

1) Добавление пустого `__init__.py`
Делает из директории пакет
Относится к Python и описывает пакет (не обязательно в современном python)

2) Mark Directory as «Sources Root»
Относится к настройке PyCharm и определяет, откуда IDE и конфигурация запуска ищут импорты



Project ▾

▾  **robopulse** ~/Desktop/github/robopulse

▾  airflow

>  dags

 test_import.py

>  docker

>  docs

>  infra

>  jupiter

>  scripts

>  spark

▾  **src**

▾  hi_scripts

 __init__.py

 hello_methods.py

>  venv

 .gitignore

 CODEX_PROJECT_CONTEXT.md

 Dockerfile.example

 generate_data.py

 Makefile

 process_silver.py

 README.md

 requirements.txt

>  External Libraries

 hello_methods.py ×

```
1 def hi1(person_name): 2 usages
2     print(f'Зanyck hello_methods {__name__}')
3     return f'Hello, {person_name}'
4
5
6 print(f'Зanyck импортированного файла {__name__}')
7
```

 test_import.py ×

```
1 from hi_scripts.hello_methods import hi1
2
3
4 print(f'Зanyck обычного файла {__name__}')
5
6 print(f'Зanyck test_import {__name__}')
7 print(hi1('Kirill'))
8
```

Run  test_import ×



```
/Users/ksysoev/Desktop/github/robopulse/venv/bin/python /Users/ksysoev/Desktop/github/robopulse/airflow/test_import.py
Зanyck импортированного файла hi_scripts.hello_methods
Зanyck обычного файла __main__
Зanyck test_import __main__
Зanyck hello_methods hi_scripts.hello_methods
Hello, Kirill

Process finished with exit code 0
```

Что можно почитать? (Исключения)

- <https://pythonworld.ru/tipy-dannyx-v-python/isklyucheniya-v-python-konstrukciya-try-except-dlya-obrabotki-isklyuchenij.html>
- <https://habr.com/ru/companies/wunderfund/articles/736526/>
- <https://metanit.com/python/tutorial/2.11.php>

Что можно почитать? (Модуль)

- <https://pythonworld.ru/moduli>
- <https://habr.com/ru/articles/718828/>
- <https://younglinux.info/oopython/module>

Спасибо за внимание!



t.me/TheDataForge



СЫСОВЕВ КИРИЛЛ

t.me/KRSysoev

